

WEEK 3 · OVERFITTING & REGULARIZATION

模型背下來了，還是學會了？

CNN 把 training accuracy 練到 99% 之後 —— 過擬合的診斷，以及三種標準對策：Data Augmentation、Dropout、Weight Decay

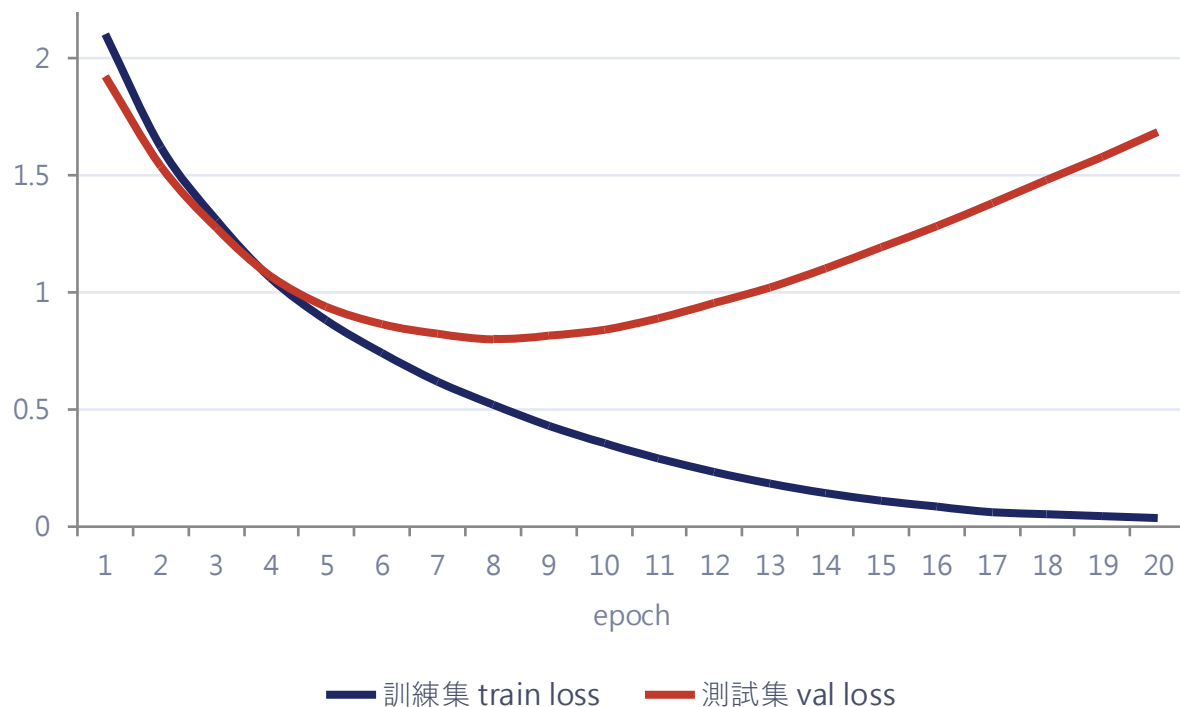
① 診斷 overfitting

② Data Augmentation

③ Dropout

④ Weight Decay

訓練 loss 一直降，測試 loss 卻轉頭向上



示意圖：CIFAR-10 上一個沒有任何正則化的 CNN。第 8 個 epoch 之後兩條線分岔 —— 那之後的訓練都在「背資料」。

01

train loss ↓ · val loss ↑

兩條曲線的分岔點，就是模型從「學規律」轉成「背資料」的那一刻。

02

train / test accuracy 出現 gap

訓練 99%、測試 68% —— 這 31 個百分點就是過擬合的量測值。

03

對輸入的小擾動極度敏感

把測試圖平移 2 個 pixel 或加一點噪音，準確率就明顯掉。

關鍵字：training loss 低，本身不是好消息。唯一有意義的指標是「沒看過的資料」上的表現 —— 所以必須切出 validation set。

為什麼會發生

網路的容量，足以「背下」整個訓練集

一個著名的實驗 (Zhang et al., 2017)

把 CIFAR-10 的 50,000 個標籤「完全打亂成隨機」，再拿同一個 CNN 去訓練 —— training accuracy 一樣可以練到接近 100%。既然標籤是隨機的，就沒有任何規律可學，模型只可能是把每一張圖硬記下來。這證明：網路的容量遠遠超過「學會規律」所需，它隨時有能力改成「查表」。

所以問題不是「模型不夠強」，而是「模型太強、資料太少」。三個施力點：

① 把資料變多

Data Augmentation

在不改變標籤的前提下，對每張訓練圖做隨機變形，等於憑空生出更多樣本。

施力在「資料」這一端

② 限制模型能用的路徑

Dropout

訓練時隨機關掉一部分神經元，逼模型不能只依賴少數幾條捷徑。

施力在「網路結構」這一端

③ 限制權重能長多大

Weight Decay

在 loss 裡加上權重平方的懲罰，讓決策邊界維持平滑。

施力在「參數大小」這一端

這三件事可以同時用，而且通常一起用 —— 它們限制的是不同的東西。

把你要的「不變性」直接教給模型



原圖



隨機裁切



水平翻轉



小角度旋轉



亮度抖動

同一張圖，每個 epoch 都長得不一樣

變形是在 DataLoader 每次取樣時即時隨機做的，所以模型幾乎不會看到兩張完全相同的輸入。

```
# 只加在 train set 上, test set 保持原樣
train_tf = transforms.Compose([
    transforms.RandomCrop(32, padding=4),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean, std),
])
test_tf = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean, std)])
```

為什麼有效

Conv 只給了你平移不變性。旋轉、縮放、亮度變化這些不變性，網路並不知道——augmentation 就是用資料把它們示範出來。

等於放大了資料集

5 萬張圖 × 每個 epoch 一組隨機變形，模型實際看到的樣本多樣性遠大於 5 萬。

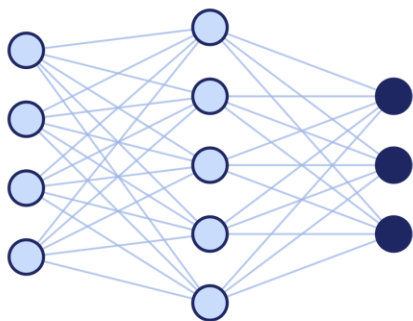
幾乎沒有成本

不需要多收一張圖，也不增加任何參數，只多花一點 CPU 前處理時間。

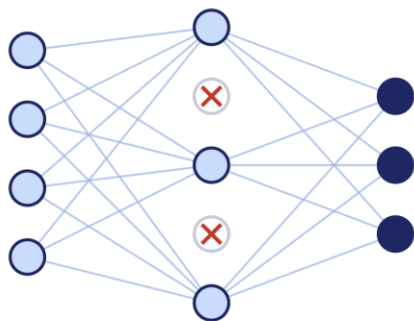
陷阱：變形必須保留標籤語意。CIFAR-10 的貓翻轉還是貓；但 MNIST 不能水平翻轉（翻過來的 2 不是 2），也不能旋轉 180°（6 會變 9）。

Dropout：訓練時隨機關掉一部分神經元

完整網路（測試時）



隨機丟棄後（訓練時）



每一個 batch 都重新抽一次遮罩 —— 所以網路的形狀每一步都不一樣。

放在哪一層？

`nn.Dropout(0.5)` 放在 Flatten 之後的全連接層 —— 參數最多、最容易背資料的就是這裡。

Conv 層不建議用一般 dropout：相鄰像素高度相關，關掉一個點的資訊還在旁邊，效果很弱。要正則化 conv 層改用 BatchNorm，或用很小的 p (0.1)。

Inverted dropout (PyTorch 的做法)

訓練： $h' = (m \odot h) / (1 - p)$

$m \sim \text{Bernoulli}(1 - p)$

$E[h'] = (1-p) \cdot h / (1-p) = h$

測試： $h' = h$ ← 什麼都不用改

先除以 $(1-p)$ 把期望值補回來，推論階段就不必再乘任何係數。

為什麼有效

神經元不能假設某個特定的同伴一定在，因此無法靠「少數幾條捷徑」硬記答案，被迫學出冗餘而分散的表徵。

另一種解釋：集成

每次抽到的子網路都不同，等於同時訓練指數多個共享權重的小網路；測試時用完整網路，近似於對它們取平均。

陷阱：一定要切 `model.train()` / `model.eval()`。忘了 `eval()` 就是測試時還在隨機丟棄。

Weight Decay：每一步都把權重往 0 拉一點

三行推導

① 在 loss 加一項懲罰：

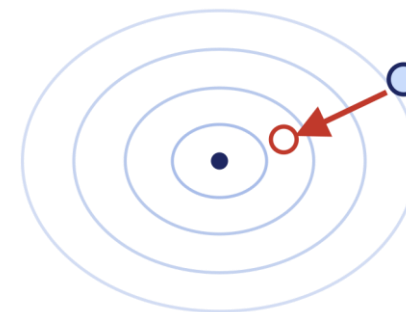
$$L_{\text{total}} = L_{\text{data}} + (\lambda/2) \cdot \sum w^2$$

② 對 w 微分（懲罰項的導數就是 λw ）：

$$\partial L_{\text{total}} / \partial w = \partial L_{\text{data}} / \partial w + \lambda w$$

③ 代回 SGD 的更新式：

$$\begin{aligned} w &\leftarrow w - \eta (\partial L_{\text{data}} / \partial w + \lambda w) \\ &= (1 - \eta \lambda) \cdot w - \eta \cdot \partial L_{\text{data}} / \partial w \end{aligned}$$



等高線 = $\|w\|^2$ 的大小。懲罰項在每一步把權重往原點拉。

(1 - $\eta\lambda$) 就是「decay」這個名字的來源：每一步先把權重乘上一個略小於 1 的數縮小一點，再走梯度。沒有梯度推它的權重，就會一路衰減到 0。

```
# 5e-4 是 CIFAR-10 上最常見的起手值
optimizer = optim.SGD(model.parameters(), lr=0.01,
                       momentum=0.9, weight_decay=5e-4)
```

直覺：小權重 = 平滑的函數

權重越大，輸入動一點點、輸出就跳很多——那正是「記住個別樣本」需要的尖銳決策邊界。壓住權重的大小，等於逼模型只能用平滑的函數解釋資料。

陷阱：上面的等價只在 SGD 成立。Adam 的自適應學習率會把 λw 一起縮放，所以要用 **AdamW**。

同一個 CNN，五種設定——自己跑一次

設定	程式改動	Train Acc	Test Acc	Gap
baseline	part3_cnn_cifar10.py 原樣	—	—	—
+ Data Augmentation	RandomCrop + HorizontalFlip	—	—	—
+ Dropout	nn.Dropout(0.5) 於 FC 層	—	—	—
+ Weight Decay	weight_decay=5e-4	—	—	—
全部一起	以上三項同時開啟	—	—	—

↑ 空欄留給你跑完自己填——重點不是絕對數字，而是 Gap 欄一路縮小。

- 1

低 training loss 不是成績

衡量模型的唯一方式是它在沒看過的資料上的表現。訓練迴圈裡一定要有 validation。
- 3

三種對策施力點不同，所以可以疊加

Augmentation 動資料、Dropout 動結構、Weight Decay 動參數大小。

- 2

過擬合來自容量與資料量的落差

網路有能力硬記整個訓練集（隨機標籤實驗）。正則化就是刻意讓「硬記」這條路變難走。
- 4

每一種都有一個必踩的坑

augmentation 要保留標籤語意；dropout 要記得 eval()；weight decay 在 Adam 上要換 AdamW。